

Analysis of Loops

1) **O(1)**: Time complexity of a function (or set of statements) is considered as O(1) if it doesn't contain loop, recursion and call to any other non-constant time function.

```
// set of non-recursive and non-loop statements
```

For example swap() function has O(1) time complexity.

A loop or recursion that runs a constant number of times is also considered as O(1). For example the following loop is O(1).

```
// Here c is a constant
for (int i = 1; i <= c; i++) {
    // some O(1) expressions
}
```

2) **O(n)**: Time Complexity of a loop is considered as O(n) if the loop variables is incremented / decremented by a constant amount. For example following functions have O(n) time complexity.

```
// Here c is a positive integer constant
for (int i = 1; i <= n; i += c) {
    // some O(1) expressions
}
for (int i = n; i > 0; i -= c) {
    // some O(1) expressions
}
```

3) **O(n^c)**: Time complexity of nested loops is equal to the number of times the innermost statement is executed. For example the following sample loops have O(n²) time complexity

```
for (int i = 1; i <= n; i += c) {
    for (int j = 1; j <= n; j += c) {
        // some O(1) expressions
    }
}
```

```
for (int i = n; i > 0; i -= c) {
    for (int j = i+1; j <= n; j += c) {
        // some O(1) expressions
    }
}
```

4) **O(Logn)**: Time Complexity of a loop is considered as O(Logn) if the loop variables is divided / multiplied by a constant amount.

```
for (int i = 1; i <= n; i *= c) {
    // some O(1) expressions
}
for (int i = n; i > 0; i /= c) {
    // some O(1) expressions
}
```

Combining Time complexities of consecutive loops:

When there are consecutive loops, we calculate time complexity as sum of time complexities of individual loops.

```
for (int i = 1; i <= m; i += c) {  
    // some O(1) expressions  
}  
for (int i = 1; i <= n; i += c) {  
    // some O(1) expressions  
}
```

Time complexity of above code is $O(m) + O(n)$ which is $O(m+n)$

If $m = n$, the time complexity becomes $O(2n)$ which is $O(n)$.

Calculating time complexity when there are many if, else statements inside loops

As discussed, worst case time complexity is the most useful among best, average and worst. Therefore we need to consider worst case. We evaluate the situation when values in if-else conditions cause maximum number of statements to be executed.

Big-O Examples

Big-O notation is used to classify running-time functions. If $f(n)$ is $O(g(n))$ then, informally, $f(n)$ is within a constant factor of $g(n)$.

1. $f(n) = 10n + 5$ and $g(n) = n$

$f(n)$ is $O(g(n))$

To show $f(n)$ is $O(g(n))$ we must show constants c and k such that

$$f(n) \leq cg(n) \text{ for all } n \geq k$$

$$\text{or: } 10n + 5 \leq cn \text{ for all } n \geq k$$

We are allowed to choose c and k to be integers we want as long as they are positive. They can be as big as we want, but they can't be functions of n .

Try $c = 15$. Then we need to show: $10n + 5 \leq 15n$.

Solving for n we get: $5 \leq 5n$ or $1 \leq n$.

So $f(n) = 10n + 5 \leq 15g(n)$ for all $n \geq 1$. ($c = 15$, $k = 1$).

Therefore we have shown $f(n)$ is $O(g(n))$.

2. $f(n) = 3n^2 + 4n + 1$.

Show $f(n)$ is $O(n^2)$

we know that:

$$4n \leq 4n^2 \text{ for all } n \geq 1$$

$$\text{and } 1 \leq n^2 \text{ for all } n \geq 1$$

$$\text{so } 3n^2 + 4n + 1 \leq 3n^2 + 4n^2 + n^2 \text{ for all } n \geq 1$$
$$\leq 8n^2 \text{ for all } n \geq 1$$

So we have shown $f(n) \leq 8n^2$ for all $n \geq 1$

so $f(n)$ is $O(n^2)$. ($c = 8$, $k = 1$)

Also try:

1. $5n+6-3n$
2. $n(n+1)/2$
3. $n^2 + n\log n + 3n$
4. $\log n!$
5. $1 + 2 + \dots + n$
6. $10000n + 1000$
7. $4n^3 + 2n^2 + 3n + 4$
8. $10n^{100}$
9. $2^n + 2^{n-1}$
10. $4n + 5/n$

Running time for popular problems

For each of the following problems, give the (best possible) worst-case running time in big-O notation. No justification needed.

1. Adding n numbers
2. Finding the min of n numbers
3. Searching in an ordered list of n items
4. Searching in an unordered list of n items
5. Sorting n items
6. Merging two lists each having n items
7. Finding the median.